

AUTOMATE TASKS WITH PYTHON + AI

O'Reilly Live Training — Quick Reference Cheatsheet

Python Fundamentals · AI APIs (OpenAI · Anthropic · Ollama) · Automation Patterns

1 · PYTHON DATA TYPES & VARIABLES

Core Data Types

Type	Example	Notes
int	age = 30	Whole numbers
float	price = 9.99	Decimal numbers
str	name = "Alice"	Text
bool	done = True	True / False
list	[1, 2, 3]	Ordered, mutable
dict	{"k": "v"}	Key-value pairs

Variables & F-Strings

```
name = "Ada"
age = 28

# F-string formatting
print(f"Hello, {name}!")
print(f"Age: {age}")

# Type conversion
num = int("42")          # str → int
pi = float("3.14")       # str → float
s = str(100)             # int → str
```

String Operations

```
s = "Hello, World!"
s.upper()          # "HELLO, WORLD!"
s.lower()          # "hello, world!"
s.strip()          # remove whitespace
s.split(", ")      # ["Hello", "World!"]
s.replace("o", "0") # "Hello, W0rld!"
len(s)             # 13
s[0:5]             # "Hello" (slice)
```

2 · FUNCTIONS

Defining & Calling

```
# Basic function
def greet(name):
    return f"Hello, {name}!"

# Default parameters
def greet(name, greeting="Hi"):
    return f"{greeting}, {name}!"

# Multiple return values
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 4, 1, 5])
print(greet("Bob"))          # Hi, Bob!
print(greet("Ana", "Hello")) # Hello, Ana!
```

Useful Built-ins

```
len(x)
Length of list/string/dict

range(n)
Range of integers 0 to n-1

type(x)
Returns the type of x

input("msg")
Read user input as string

sorted(iterable)
Return new sorted list

enumerate(lst)
Index + value pairs

zip(a, b)
Pair elements from two lists
```

3 · LISTS & LOOPS

List Operations

```
fruits = ["apple", "banana", "cherry"]

fruits.append("mango")      # add to end
fruits.insert(1, "kiwi")    # insert at index
fruits.remove("banana")     # remove by value
popped = fruits.pop()       # remove last
fruits.sort()               # sort in place
fruits.reverse()            # reverse in place

# Access
fruits[0]                   # first element
fruits[-1]                  # last element
fruits[1:3]                  # slice [start:end]
len(fruits)                  # count
```

For Loops & List Comprehensions

```
# Basic for loop
for item in fruits:
    print(item)

# Loop with index
for i, item in enumerate(fruits):
    print(f"{i}: {item}")

# Loop over range
for i in range(5):          # 0,1,2,3,4
    print(i)

# List comprehension
squares = [x**2 for x in range(10)]
evens    = [x for x in range(20) if x%2==0]
upper    = [s.upper() for s in fruits]
```

4 · DICTIONARIES

Dictionary Basics

```
person = {"name": "Alice", "age": 30}

# Access
person["name"]          # "Alice"
person.get("email", "N/A") # safe get

# Modify
person["email"] = "a@b.com" # add/update
del person["age"]           # delete key

# Methods
person.keys()            # all keys
person.values()           # all values
person.items()            # key-value pairs

# Loop
for key, val in person.items():
    print(f"{key}: {val}")
```

Nested Dictionaries

```
users = {
    "alice": {"age": 30, "role": "admin"},
    "bob":   {"age": 25, "role": "user"},
}
users["alice"]["role"] # "admin"
users["bob"]["age"] = 26 # update nested

# Create from loop
scores = {name: 0 for name in ["a", "b", "c"]}
```

5 · CONDITIONALS & LOGIC

If / Elif / Else

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

# One-liner (ternary)
status = "pass" if score >= 60 else "fail"
```

Comparison & Boolean Operators

Op	Meaning	Example
==	Equal	x == 5
!=	Not equal	x != 0
>, <	Greater/Less	x > 10
>=, <=	Grtr/Less or eq	x <= 100
and	Both true	a and b
or	Either true	a or b
not	Negation	not done
in	Membership	"x" in lst

6 · FILE I/O

Reading & Writing Files

```
# Read a text file (always use 'with')
with open("notes.txt", "r") as f:
    content = f.read()      # entire file
    # lines = f.readlines() # list of lines

# Write (creates or overwrites)
with open("output.txt", "w") as f:
    f.write("Hello, file!\n")

# Append to existing file
with open("log.txt", "a") as f:
    f.write("New entry\n")
```

File Modes

Mode	Meaning
"r"	Read (file must exist)
"w"	Write (creates/overwrites)
"a"	Append to end of file
"r+"	Read and write

File + AI Pattern

```
# Read → AI summarize → write result
with open("article.txt", "r") as f:
    text = f.read()

summary = ask_ai(f"Summarize:\n{text}")

with open("summary.txt", "w") as f:
    f.write(summary)
```

7 · CSV & PANDAS

Reading & Filtering CSV

```
import pandas as pd

df = pd.read_csv("data.csv")
df.head()          # first 5 rows
df.info()          # column types
df.describe()      # stats summary

# Filter rows
high = df[df["priority"] == "High"]
cheap = df[df["price"] < 50]

# Select columns
df[["name", "price"]]

# Add a column
df["tax"] = df["price"] * 0.1
```

Creating & Exporting

```
import csv

# Write CSV with csv module
data = [
    ["Name", "Age"],
    ["Alice", 30]
]
with open("people.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)

# Export DataFrame
df.to_csv("output.csv", index=False)
df.to_json("output.json", orient="records")
```

8 · PACKAGES & APIS

Package Management

```
# Install packages
pip install requests pandas openai anthropic

# With uv (this course)
uv sync          # install from pyproject.toml
uv run python main.py # run a script

# Jupyter Lab
uv run --with jupyter jupyter lab
```

Requests & REST APIs

```
import requests, os

API_KEY = os.getenv("MY_API_KEY")

response = requests.get(
    "https://api.example.com/data",
    headers={"Authorization": f"Bearer {API_KEY}"},
    params={"q": "python", "limit": 10}
)
data = response.json()
print(response.status_code) # 200 = OK
```

Environment Variables

```
# .env file (never commit to git!)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...

# Load in Python
from dotenv import load_dotenv
import os

load_dotenv()
key = os.getenv("OPENAI_API_KEY")
```

9 · AI APIS — OPENAI

Chat Completions

```
from openai import OpenAI

client = OpenAI() # reads OPENAI_API_KEY

response = client.chat.completions.create(
    model="gpt-4o",          # or "gpt-4o-mini"
    messages=[
        {"role": "system",
         "content": "You are a helpful assistant."},
        {"role": "user",
         "content": "Explain recursion simply."}
    ]
)
text = response.choices[0].message.content
```

Image Generation (DALL-E 3)

```
response = client.images.generate(
    model="dall-e-3",
    prompt="A futuristic Python logo, digital art",
    size="1024x1024",
    quality="standard",
    n=1
)
image_url = response.data[0].url
```

Audio Transcription (Whisper)

```
with open("recording.mp3", "rb") as audio:
    transcript = client.audio.transcriptions.create(
        model="whisper-1",
        file=audio
    )
print(transcript.text)
```

OpenAI Models Reference

Model	Best For
<code>gpt-4o</code>	Complex tasks, vision
<code>gpt-4o-mini</code>	Fast, cheap, everyday use
<code>dall-e-3</code>	Image generation
<code>whisper-1</code>	Audio transcription

10 · AI APIS — ANTHROPIC (CLAUDE)

Basic Message

```
import anthropic

client = anthropic.Anthropic() # reads
ANTHROPIC_API_KEY

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {"role": "user",
         "content": "Summarize this: ..."}
    ]
)
text = response.content[0].text
```

With System Prompt

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=2048,
    system="You are an expert Python tutor.",
    messages=[
        {"role": "user",
         "content": "Explain list comprehensions."}
    ]
)
```

Claude Models

Model	Use Case
claude-opus-4-7	Most capable, complex reasoning
claude-sonnet-4-6	Balanced speed & quality
claude-haiku-4-5	Fastest, lightest tasks

11 · AI APIS — OLLAMA (LOCAL MODELS)

Running Local Models

```
import ollama

# Simple text generation
response = ollama.chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "Hello!"}]
)
print(response.message.content)

# Using Client class
client = ollama.Client()
resp = client.chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "Explain
AI"}]
)
```

Structured JSON Output

```
resp = ollama.chat(
    model="llama3.2",
    messages=[{
        "role": "user",
        "content": "List 3 Python tips as JSON"
    }],
    format="json"
)
import json
data = json.loads(resp.message.content)
```

Setup Ollama

Install from ollama.com, then run: `ollama pull llama3.2` to download a model. Runs 100% locally — no API key needed.

12 · UNIFIED AI INTERFACE (AI_TOOLS.PY)

The ask_ai() Function

```
from scripts.ai_tools import ask_ai, ask_local_ai

# Cloud AI — OpenAI
result = ask_ai("Summarize: ...",
                model_name="gpt-4o-mini")

# Cloud AI — Anthropic
result = ask_ai("Review code: ...",
                model_name="claude-sonnet-4-6")

# Local AI via Ollama
result = ask_local_ai("Explain decorators",
                      model_name="llama3.2")

# Structured JSON from local AI
data = ask_local_ai(
    "Return invoice data as JSON: ...",
    model_name="llama3.2",
    structured=True
)
```

parse_json_output()

```
from scripts.ai_tools import parse_json_output

raw = ask_ai(
    "Return as JSON: {name, price, qty} for 3 items"
)
# Strips markdown fences, parses JSON
data = parse_json_output(raw)
print(data["items"][0]["name"])
```

Prompt Templates

```
def summarize(text, max_words=100):
    prompt = f"""
    Summarize the following in {max_words} words.
    Focus on key points only.

    TEXT:
    {text}

    SUMMARY: """
    return ask_ai(prompt)

def categorize(item, categories):
    cats = ", ".join(categories)
    prompt = f"""
    Categorize into one of: {cats}
    Item: {item}
    Return only the category name. """
    return ask_ai(prompt)
```

Prompt Chaining

```
# Step 1: Generate idea
idea = ask_ai("Suggest a creative image prompt")

# Step 2: Critique it
critique = ask_ai(
    f"Critique and suggest improvements:\n{idea}"
)

# Step 3: Refine
refined = ask_ai(
    f"Original: {idea}\nFeedback: {critique}\n"
    f"Write an improved version:"
)

# Step 4: Generate image
img = client.images.generate(
    model="dall-e-3", prompt=refined)
```

Structured Output Prompt

```
prompt = """
Extract invoice data as JSON with this schema:
{
  "vendor": "string",
  "total": number,
  "date": "YYYY-MM-DD",
  "items": [{"desc": "string", "price": number}]
}

Invoice text:
{invoice_text}

Return only valid JSON, no markdown. """
```

Tip: Always specify the exact output format. Say "Return only JSON, no explanation" to avoid markdown fences you'd need to strip manually.

14 · AUTOMATION PROJECT PATTERNS

File Organizer

```
import shutil
from pathlib import Path

folder = Path("./downloads")
rules = {
    ".pdf": "docs", ".jpg": "images",
    ".png": "images", ".csv": "data"
}

for file in folder.iterdir():
    if file.is_file():
        dest_dir = folder / rules.get(file.suffix,
"other")
        dest_dir.mkdir(exist_ok=True)
        shutil.move(str(file), dest_dir / file.name)
```

PDF Summarizer

```
import PyPDF2

def summarize_pdf(path):
    with open(path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        text = ""
        for page in reader.pages:
            text += page.extract_text()
        return ask_ai(f"Summarize:\n{text[:4000]}")

summary = summarize_pdf("paper.pdf")
```

Invoice / Data Extractor

```
def extract_invoice(text):
    prompt = f"""
    Extract as JSON: vendor, total, date, items.
    Text: {text}
    Return valid JSON only."""
    return parse_json_output(ask_ai(prompt))

# Batch process a folder
import os
results = []
for fname in os.listdir("invoices/"):
    with open(f"invoices/{fname}") as f:
        results.append(extract_invoice(f.read()))
```

Data Analysis Pipeline

```
import pandas as pd

df = pd.read_csv("sales.csv")
stats = df.describe().to_string()

# AI interprets the stats
analysis = ask_ai(
    f"Analyze stats, give 3 insights:\n{stats}"
)

# Categorize rows with AI
df["category"] = df["description"].apply(
    lambda x: ask_ai(
        f"Category for: {x}. "
        f"Options: tech/finance/other. One word."
    )
)
df.to_csv("sales_categorized.csv", index=False)
```

15 · WEB SCRAPING

Requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com/articles"
resp = requests.get(url)
soup = BeautifulSoup(resp.text, "html.parser")

# Find elements
title = soup.find("h1").text
links = [a["href"] for a in soup.find_all("a")]
articles = soup.select(".article-card")
```

Playwright (Browser Automation)

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://example.com")
    page.fill("#search", "Python")
    page.press("#search", "Enter")
    page.wait_for_selector(".results")
    results = page.query_selector_all(".result")
    texts = [r.inner_text() for r in results]
    browser.close()
```

Process Multiple Files with AI

```
import os

results = {}
for filename in os.listdir("docs/"):
    if filename.endswith(".txt"):
        with open(f"docs/{filename}") as f:
            text = f.read()
            results[filename] = ask_ai(
                f"One-sentence summary: {text}"
            )

# Write index file
with open("index.md", "w") as f:
    for name, summary in results.items():
        f.write(f"- **{name}**: {summary}\n")
```

Common Imports

Import	Use For
<code>import os</code>	Paths, env vars, dirs
<code>import json</code>	Parse / create JSON
<code>import csv</code>	Read/write CSV files
<code>import shutil</code>	Move/copy/delete files
<code>from pathlib import Path</code>	Modern path handling
<code>import pandas as pd</code>	DataFrames / tabular data
<code>import requests</code>	HTTP / REST APIs
<code>from openai import OpenAI</code>	OpenAI chat/image/audio
<code>import anthropic</code>	Claude API
<code>import ollama</code>	Local LLMs

UV COMMANDS (THIS COURSE)

```
| uv sync
  Install all project deps

| uv run python script.py
  Run a Python script

| uv run --with jupyter jupyter lab
  Start Jupyter

| playwright install
  Install browser binaries
```

Setup: Copy `.env.example` → `.env` and add your `OPENAI_API_KEY` and `ANTHROPIC_API_KEY` before running AI notebooks.